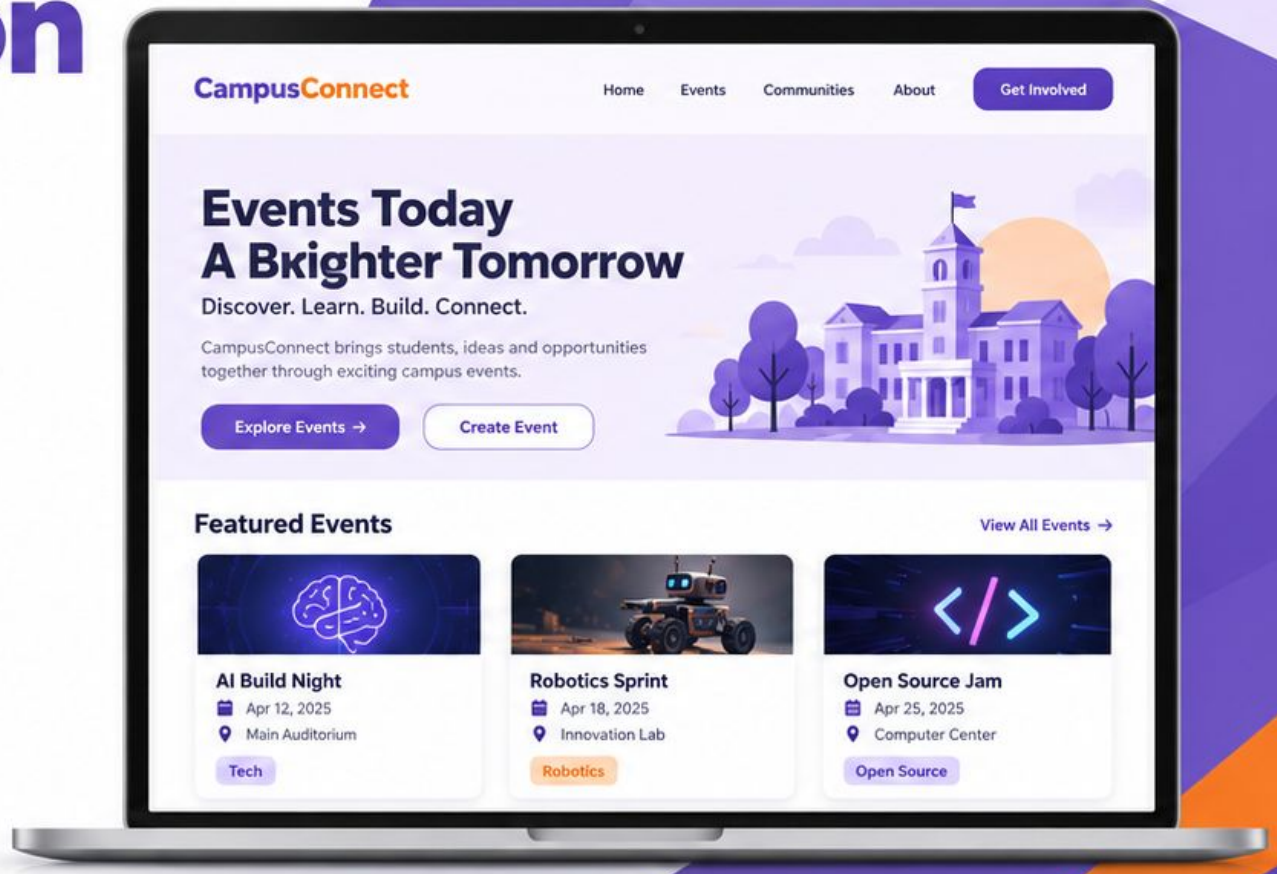
IntelliPaat

From Business Idea To Working Application

Day 1 | CampusConnect

- One Client Challenge
- Five Professional Roles
- One Working Product



Discover → Architect → Build → Design → Connect

Your Role In Building The Application

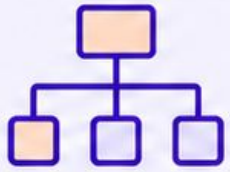
Explore five professional roles, from defining requirements to building the application.



Product Manager & Business Analyst

Understand users, define scope and specify requirements.

Output:
Product Specification



System Architect

Define the system structure, data model and API contracts.

Output:
Technical Design



Backend Engineer

Implement APIs, business rules and data storage.

Output:
Tested Backend



UI/UX Designer

Design user journeys, screens and interaction states.

Output:
UI/UX Designs



Frontend Engineer

Build the interface and connect it to the APIs.

Output:
Connected Application

Each role builds on shared decisions and feedback.

© Copyright Intellipaat. All rights reserved.

Understand The Client Challenge

Your team has been asked to improve how a college manages campus events.

Current Process

Forms

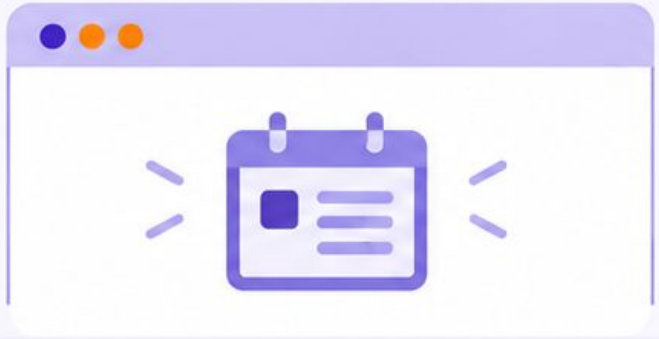
Spreadsheets

Group Chats

Students struggle to find event details and confirmation.

Organizers struggle to track registrations and available seats.

Client Request



> We need one place to share campus events and manage registrations.

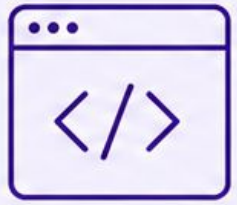
Before we build, what do we need to clarify?

© Copyright Intellipaat. All rights reserved.

Poll: Where Should We Start?

The client has shared the challenge. What should your team do first?

A  Design The Screens

B  Build The Backend

C  Choose A Database

D  Interview Stakeholders

Choose one option and explain why it should come first.

© Copyright Intellipaat. All rights reserved.

Think Like A Product Manager & Business Analyst

Your Mission As The Product Manager

IntelliPaat

Start by understanding users and their needs. Define the scope and turn those needs into clear requirements.

1



Understand The Problem

What problem are we solving, and why does it matter?

2



Identify User Needs

Who will use the application, and what do they need to accomplish?

3



Define The Scope

What should the first version include, and what can wait?

4



Specify Requirements

How should each feature behave, and how will we verify it works?

Your Output: A product specification that guides design, architecture and development.

© Copyright Intellipaat. All rights reserved.

Identify Stakeholders And End Users

Start by identifying who will use the application and who influences its requirements.

Stakeholder

Anyone affected by the product or involved in its decisions.

End User

Someone who directly uses the application to complete a task.

Who Matters For CampusConnect?



Students

Discover relevant events and register.

End users and stakeholders



Organizers

View accurate attendee lists for their own events.

End users and stakeholders



Campus Administration

Define campus policies and oversight needs.

Stakeholder; tools deferred

ⓘ A person can be both a stakeholder and an end user.

© Copyright Intellipaat. All rights reserved.

Understand Each User's Goals And Challenges

The Product Manager uses personas to understand user needs and guide feature decisions.



Student

Goal

Find relevant events and know that registration is confirmed.

Current Challenge

Event information is scattered and registration status is unclear.



Organizer

Goal

Access an accurate attendee list for an event they own.

Current Challenge

Registration records are spread across forms and spreadsheets.



Campus Administrator

Goal

Understand campus activity and ensure events follow policy.

Current Challenge

Information is fragmented across organizers.

These initial assumptions will be validated during the client interview.

MVP Focus: Student journey and organizer attendee access. Administrator tools are deferred.

© Copyright Intellipaat. All rights reserved.

Map The Current User Journey

The Product Manager examines how users complete tasks today to identify problems worth solving.

Current Process: Before CampusConnect

					Friction To Investigate
 Student	Find an announcement	→ Locate event details	→ Submit a form	→ Look for confirmation	Missing links, unclear event information and uncertain registration status.
 Organizer	Share event information	→ Collect responses	→ Combine records	→ Prepare an attendee list	Scattered responses, duplicate entries and difficulty tracking available seats.
 Campus Administrator	Request updates	→ Collect organizer reports	→ Review activity		Incomplete information and limited visibility across events.

Next Step: Validate these problems with the client before selecting features.

© Copyright Intellipaat. All rights reserved.

Validate User Needs With The Client

Use the journey map to check your assumptions and clarify requirements.

7 Minutes

Your Task

1. Review the goals and friction points.
2. Prepare five discovery questions.
3. Interview the client and record the answers.



Trainer:
Acts as the client

Teams:
Ask questions and capture findings

Discovery Board

Five Discovery Questions

1. _____
2. _____
3. _____
4. _____
5. _____

Three User Goals

1. _____
2. _____
3. _____

Three Friction Points

1. _____
2. _____
3. _____

Two Unknowns

1. _____
2. _____

Deliverable: A completed discovery board to agree on findings before selecting features.

© Copyright Intellipaat. All rights reserved.

Confirm What We Learned From Discovery

Compare your interview findings with these agreed workshop assumptions.

Find Relevant Events

Students can browse and filter events before choosing one.

Why it matters: Students can find suitable events before registering.

Register Once

Signed-in students can register once per event.

Why it matters: Each registration belongs to an identified student, without duplicates.

Respect Event Capacity

Registration succeeds only for open events with available seats.

Why it matters: Closed or full events must not accept new registrations.

Keep Confirmation Visible

A successful registration is saved and visible in My Registrations.

Why it matters: Students can check their booking after leaving the confirmation screen.

Protect Attendee Access

Organizers see attendees only for events they own.

Why it matters: Attendee information stays within the correct event team.

Use these rules to define the user journey and compare candidate features.

Now That We Understand Our Users, What Should We Build First?

© Copyright Intellipaat. All rights reserved.

Define The Student Journey In CampusConnect

Use the validated needs to define the experience our application will support.

Proposed Application Journey

Find An Event



1 Browse Events

Explore available campus events.



2 Filter Events

Narrow the list to relevant events.



3 View Event Details

Review event information and availability.

Not signed in? Sign in, then return to the selected event.

Continue when signed in

Complete Registration



4 Register

Submit a registration for the selected event.



5 See Confirmation

Check that registration succeeded.



6 View My Registrations

Find the saved registration later.

Which capability does the application need at each step?

Identify Features That Support The Journey

Turn the problems we identified into candidate features for CampusConnect.

User Problem		Candidate Feature
Scattered announcements	→	**Browse Events**
Hard to find relevant events	→	**Search And Filters**
Unclear event facts	→	**Event Details**
Uncertain registration	→	**Confirmation**
Lost registration records	→	**My Registrations**
Organizer: Scattered attendee records	→	**Organizer Attendee List**

These are candidate features. Next, decide what belongs in the first release.

© Copyright Intellipaat. All rights reserved.

Explore Candidate Features

Review the journey-based features alongside other possibilities before choosing the first release.

Feature Pool • Not Yet Prioritized



Browse And Filter

Find relevant campus events.



Event Authoring

Create and update event information.



Sign-In

Access the app with a verified identity.



Recommendations

Suggest events based on interests.



Event Details

Review an event before registering.



Administration Tools

Review campus activity and policies.



Registration

Reserve a place in an event.



My Registrations

View saved event registrations.



Organizer Attendee Access

View attendees for owned events.



Confirmation

See whether registration succeeded.

Next: Compare user value, dependencies and delivery effort.



Which features deliver the most user value, and which can wait?

© Copyright Intellipaat. All rights reserved.

Choose The First Release

Your Team's Task

1. Complete the student registration journey.
2. Include organizer attendee access.
3. Sort the candidate features into MVP and Later.

How To Decide

User Value
Does it meet an essential user need?

Dependencies
What must exist for it to work?

Delivery Effort
Can we build it within the workshop scope?

Build Your Release Plan

MVP — Needed for the first release	
Feature	Reason

Later — Can wait	
Feature	Reason

Deliver: MVP and Later lists with reasons.
Explain one deferred feature and one essential dependency.

Next: Compare your choices with the reference backlog.

© Copyright Intellipaat. All rights reserved.

Agree On The CampusConnect MVP

Compare your choices with the shared scope for our workshop build.

First Release · 7 Features

FR-01	Sign in.
FR-02	Browse and filter events.
FR-03	View event details.
FR-04	Register for an event.
FR-05	See registration confirmation.
FR-06	View My Registrations.
FR-07	Organizer views attendees for owned events.

Build Assumptions

Events and organizer ownership are preloaded.

The first release uses prepared event data.

Deferred From This Release

- Event authoring
- Administrator tools
- Recommendations

Scope: A complete student registration journey, plus organizer attendee access.

© Copyright Intellipaat. All rights reserved.

Define The Student Stories

User Story ID	Actor	Intent (As a … I want …)	Business Value (So that …)
US-01	Student	As a student, I want to sign in to the platform (**FR-01**).	So that I can securely access event features.
US-02	Student	As a student, I want to browse and search events (**FR-02**).	So that I can find events that match my interests.
US-03	Student	As a student, I want to view event details (**FR-03**).	So that I can learn more and decide whether to register.
US-04	Student	As a student, I want to register for an event (**FR-04**).	So that I can reserve my spot in the event.
US-05	Student	As a student, I want to see a registration confirmation (**FR-05**).	So that I know my registration is successful.
US-06	Student	As a student, I want to view my registrations (**FR-06**).	So that I can track the events I have registered for.

© Copyright Intellipaat. All rights reserved.

Document The Agreed Product Scope

The Product Manager records our agreed decisions in a Business Requirements Document (BRD).

01

Problem

Event information and registration records are scattered across forms, spreadsheets and chats.

02

Users And Stakeholders

MVP users: Students and organizers.

Stakeholder: Campus administration.

03

Agreed MVP

Seven functional requirements (FR-01–FR-07): sign-in, browse and filter, event details, registration, confirmation, My Registrations and organizer attendee access.

04

Assumptions And Exclusions

Preloaded: Events and organizer ownership.

Deferred: Event authoring, administrator tools and recommendations.

05

Expected Outcomes

Students can find events, register and revisit confirmed registrations.

Organizers can access attendee lists only for events they own.

Scope baseline for detailed user stories and business rules.

© Copyright Intellipaat. All rights reserved.

We Have Defined The Scope. Now Let’s Define How Each Feature Should Work.

© Copyright Intellipaat. All rights reserved.

Verifying User Stories

Give each agreed feature a user, an action and a purpose.

Template: As a **[role]**, , I want **[action]**, , so that **[benefit]**.

	**Discover Events**	→	As a **user** ,	I want **to discover events,**	so that	**I can find events that interest me.**
	**View Event Details**	→	As a **user** ,	**I want to view event details,**	so that	**I can learn more before registering.**
	**Register for Event**	→	As a **user** ,	**I want to register for an event,**	so that	**I can secure my spot to attend.**
	**View Registration History**	→	As a **user** ,	**I want to view my registration history,**	so that	**I can keep track of the events I have registered for.**
	**Organizer Attendee Access**	→	As an **organizer** ,	**I want to access the attendee list,**	so that	**I can manage event attendees effectively.**

Define Organizer Access And Scope Boundaries

Complete the MVP story set with organizer access and clear ownership rules.

US-07 / FR-07

As an organizer, I want to view attendees for **events I own** so that I can prepare for each event.

Ownership Determines Access



Owning organizer

Event Attendees





Allowed

View the event's attendee list.



Another organizer

Event Attendees

Denied

Cannot access this attendee list.

Release Boundaries

Included

Attendee viewing for owned events.

Deferred

Event authoring
Administrator tools

Administrator oversight remains a future need.

Make Registration Rules Testable

Turn the user stories into observable acceptance criteria.

FR-04 · Registration

GIVEN
A signed-in student has not registered; the event is open and has a seat.

WHEN
The student submits a registration.

THEN
Save exactly one registration and show confirmation.



Check The Exceptions

Already registered
Do not create a duplicate; explain the existing registration.

Event full or closed
Do not register; explain why.

Not signed in
Prompt sign-in, then return to the selected event.

Event not found
Show unavailable; create no registration.

FR-07

Check Organizer Access

Given an organizer does not own the event,
when they request its attendee list,
then deny access.



Rejected registration attempts leave registration records unchanged.

Specify Registration Behaviour

Capture the agreed rules in a Functional Requirements Document (FRD).

FR-04 / US-04 | Register For An Event

Actor And Input



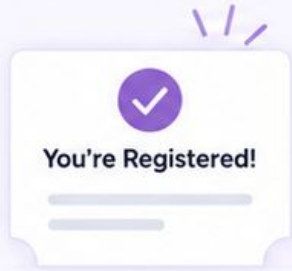
Actor: Signed-in student.

Input: Selected event; identity from the authenticated session.

Validation Checks

- Event exists
- Registration is open
- A seat is available
- Student has not already registered

On Success



Save exactly one registration.

Show confirmation.

Include it in My Registrations.

On Rejection



Missing sign-in, missing event, closed or full event, or duplicate registration.

Explain the reason; create no registration.

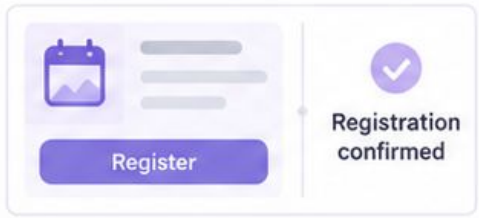
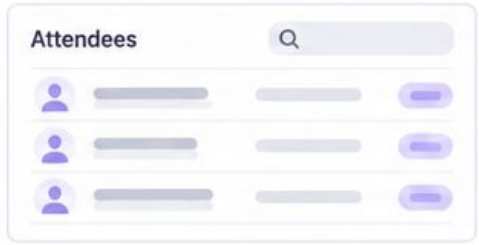
Data Integrity | Never exceed capacity or create duplicates—even when requests arrive together.

The architect and backend engineer translate these outcomes into API contracts.

One-Tailed versus Two-Tailed

Connect each agreed requirement to its implementation and verification.

Illustrative mappings · Example test cases

Requirement	User Story	Screen	API Operation	Acceptance Tests
FR-04 Register for an event	**US-04** Reserve a place	Event details and confirmation 	Register for an event	✓ Successful registration ✓ Duplicate rejected ✓ Full event rejected
FR-07 View owned-event attendees	**US-07** Prepare for the event	Organizer attendee view 	Retrieve event attendees	✓ Owner allowed ✓ Other organizer denied

Complete Coverage
Create mappings for all seven MVP requirements.

Design References To Confirm
Agree screen and API references with the designer and architect; technical IDs are not yet approved.

© Copyright Intellipaat. All rights reserved.

Our Product Requirements Are Ready. Let’s Bring Them Together For The Architect.

© Copyright Intellipaat. All rights reserved.

Review The Product Specification For Handoff

Bring the agreed decisions together before technical design begins.

The Handoff Package

A consolidated set of decisions and artefacts for the next phase.



Product Brief

Problem, users, scope, assumptions and exclusions.

Journey And MVP Backlog

Complete student journey and FR-01–FR-07.

User Stories And Acceptance Criteria

User value, permissions and testable outcomes.

Functional Specification And Traceability

Detailed behaviour and links across delivery.

Open Decisions

An owner and due point for every unresolved item.

Readiness Review

Check that the handoff package is complete and unblocked before moving to technical design.

- [ ] Scope is agreed
- [ ] Build blockers are resolved
- [ ] Accepted assumptions are recorded
- [ ] Remaining decisions have owners and due points

Ready Only After Review

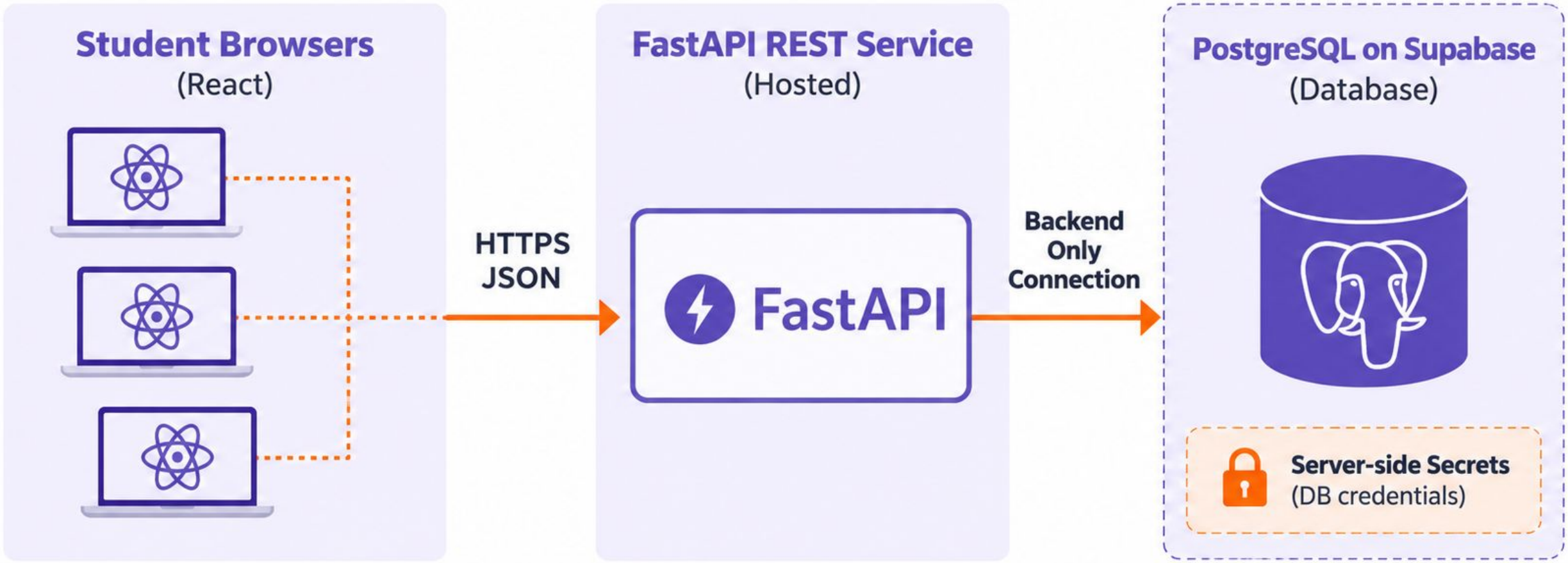
Confirm readiness for the workshop build after reviewing the package.



Product behaviour becomes system design.

Think Like A System Architect

High Level Architecture of Campus Fix

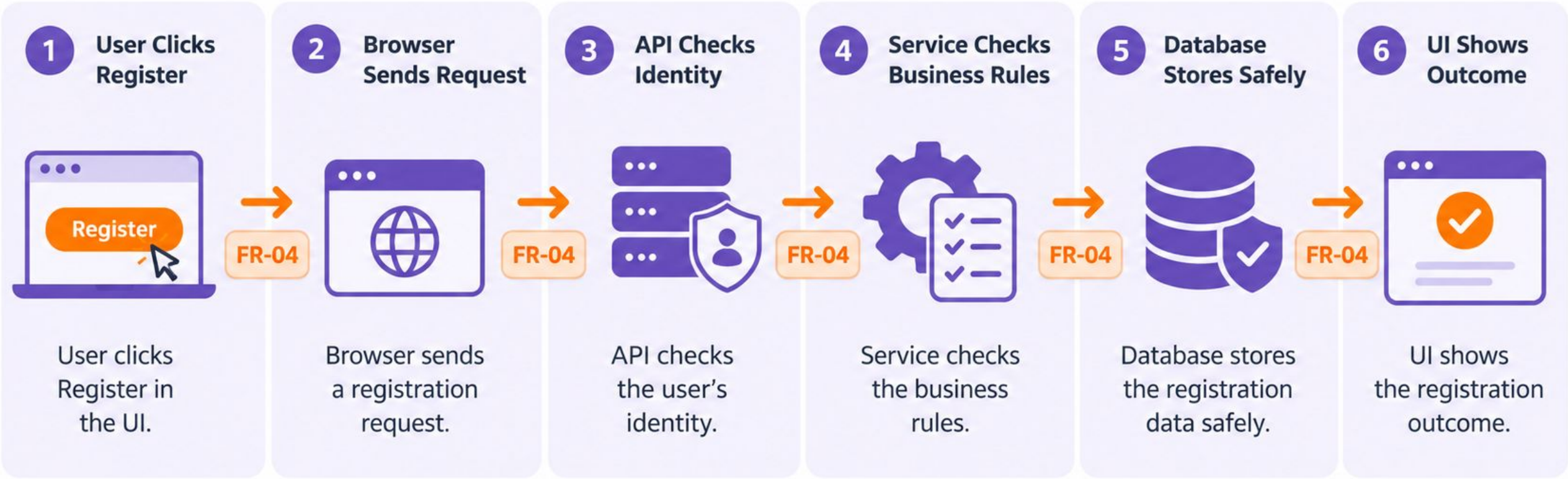


Let’s take one Function Requirement, and design the System Design for it

Let’s start with FR 04 - Register

© Copyright Intellipaat. All rights reserved.

Steps to Implement the Feature



In System Design, we don’t only architect for Functional Requirements.

Along with Functional Requirements, we also have to take care of Non Functional Requirements

© Copyright Intellipaat. All rights reserved.

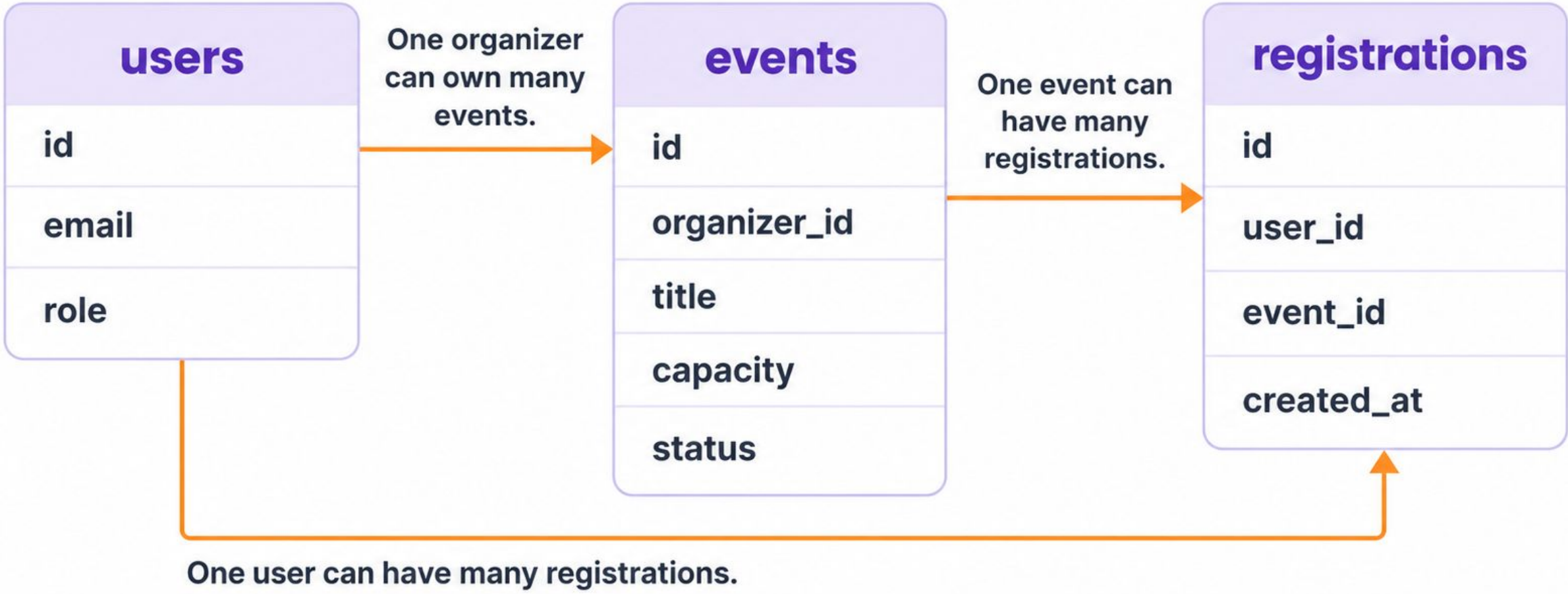
Key Non-Functional Requirements

Reliability	Prevent duplicates and overbooking.
Security	Authenticate users and enforce event ownership.
Usability	Clear feedback and keyboard access.
Performance	Agree response-time targets and test with realistic data. Targets require stakeholder agreement.
Maintainability	Separate API, business logic and persistence.

Let’s design the Database Tables for this

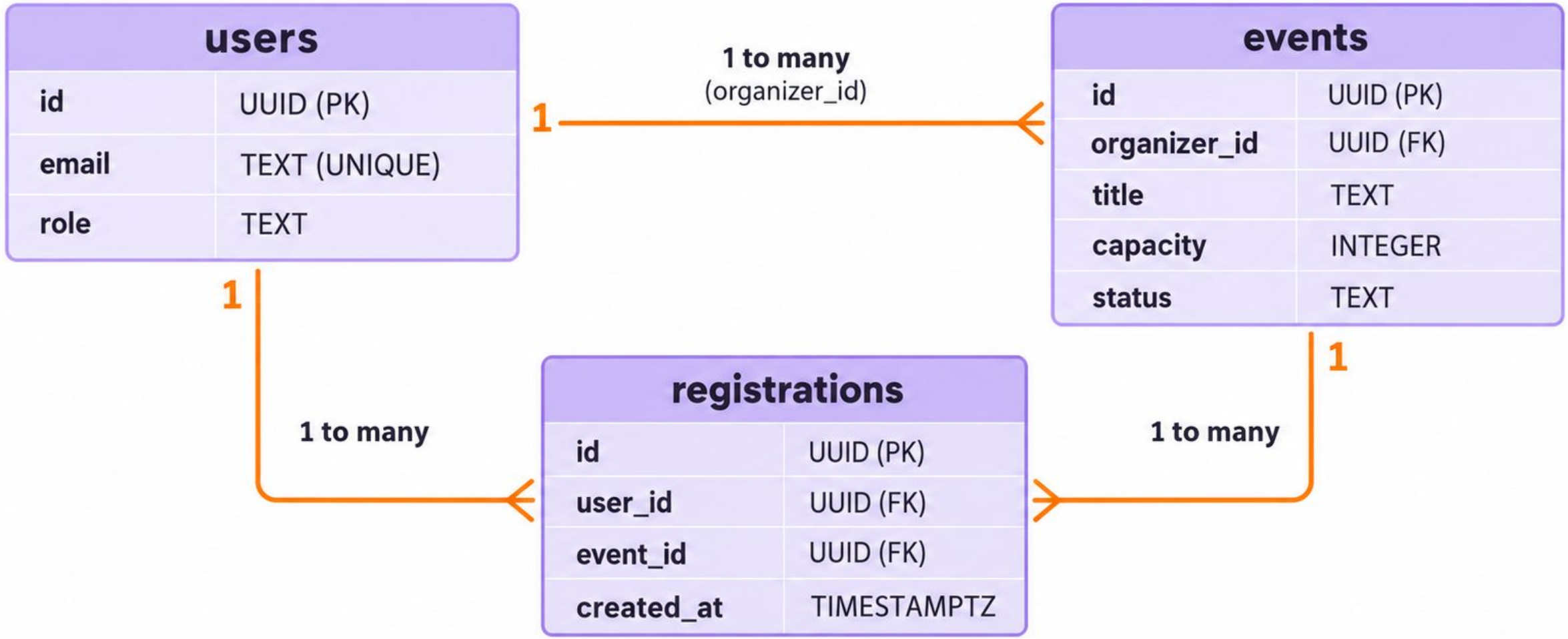
© Copyright Intellipaat. All rights reserved.

Identify the core entities



IDs use UUIDs.

Relationships And The ERD



UNIQUE(user_id, event_id) prevents duplicate registrations.
Registrations links students to events.

7 minutes. Model the data

- Model users, events and registrations.
- Mark primary keys and foreign keys.
- Show relationship cardinality.
- Add a rule to prevent duplicate registration.

Deliver: A three-entity ERD with constraints.



Schema: protect the data

users

- UUID primary key
- unique email

events

- UUID primary key
- organizer_id references users.id
- capacity greater than zero

registrations

- UUID primary key
- user_id references users.id
- event_id references events.id
- created_at timestamp with time zone
- UNIQUE(user_id, event_id)

- Application supplies UUIDs.
- Unique constraint prevents duplicates; transaction logic protects capacity.

Implement the ERD with SQL

IntelliPaat

Events Table

```
CREATE TABLE events (
    id uuid PRIMARY KEY,
    organizer_id uuid NOT NULL REFERENCES users(id),
    title text NOT NULL,
    capacity integer NOT NULL CHECK (capacity > 0),
    status text NOT NULL
);
```

- Primary key with UUID (application supplies values)
- References users table (event organizer)
- Title is required
- Capacity must be greater than 0
- Event status is required

Registrations Table

```
CREATE TABLE registrations (
    id uuid PRIMARY KEY,
    user_id uuid NOT NULL REFERENCES users(id),
    event_id uuid NOT NULL REFERENCES events(id),
    created_at timestamptz NOT NULL DEFAULT now(),
    UNIQUE (user_id, event_id)
);
```

- Primary key with UUID (application supplies values)
- References users table (registrant)
- References events table (event being registered for)
- Timestamp with default current time
- Ensures a user can register only once per event

The Last-Seat Incident



Event Capacity: 100 seats

Existing Bookings: 99 seats

Remaining Seats: 1

Only one seat remains. Students A and B query **remaining = 1** simultaneously. Both try inserting. Without safe concurrency control, the count can exceed capacity.

T1 (Student A)

Read Remaining Seats → remaining = 1

Check Availability → 1 > 0 (proceed)

Insert Booking for A → Succeeds

T2 (Student B)

Read Remaining Seats → remaining = 1

Check Availability → 1 > 0 (proceed)

Insert Booking for B → Succeeds

Final Result

Event Capacity

100

Total Bookings After Both Inserts

101

Count Exceeds Capacity (Overbooking)

MISSION — Protect the last seat

IntelliPaat

5 minutes.

Two students request the final seat at the same time.

Design a safe transaction protocol.

Explain what the second request observes.

State where rollback happens.

Deliver: ordered steps and expected outcomes.

Student A

Student B

ACID for registration

IntelliPaat



Atomicity

All transaction changes commit together or roll back.



Consistency

Constraints and correct logic preserve valid data.



Isolation

Concurrent requests follow a safe locking protocol.



Durability

Committed changes survive failures.

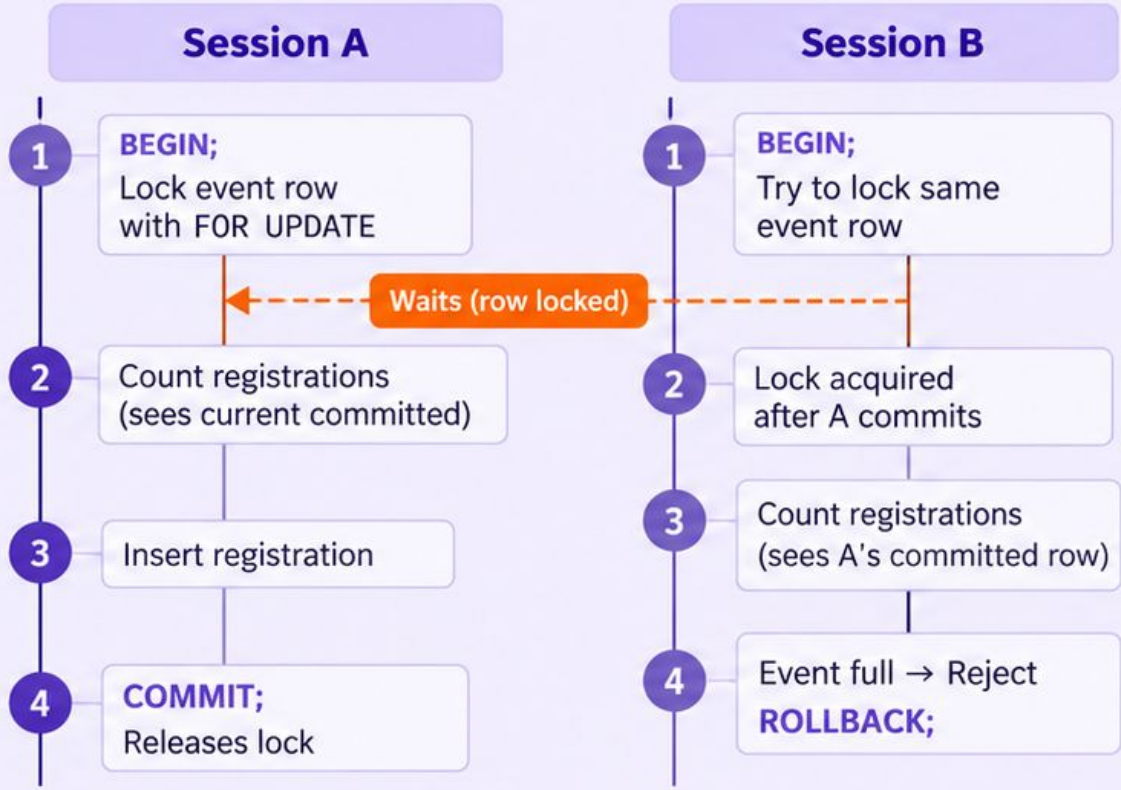


Atomicity alone does not prevent overbooking.

Safe PostgreSQL Locking Transaction

IntelliPaat

Two Concurrent Sessions



Transaction Pseudocode · READ COMMITTED

```
1   BEGIN;
2   SELECT id, capacity
3   FROM events
4   WHERE id = :event_id
5   FOR UPDATE;                    -- Acquire exclusive lock on this event row

6   SELECT COUNT(*)
7   FROM registrations
8   WHERE event_id = :event_id;    -- Count committed rows (after lock is held)

9   -- Application decision (pseudocode)
    IF count >= capacity THEN
       ROLLBACK;  -- reject if full
    ELSE
       INSERT INTO registrations (event_id, user_id, ...)
       VALUES (:event_id, :user_id, ...);
       COMMIT;
10  END IF;
```

1 All Capacity-Changing Writers Lock First

Every transaction that can change registrations must lock the same event row using `FOR UPDATE` before counting or inserting.

2 Unique Constraint Catches Same-User Races

A unique constraint on `(event_id, user_id)` prevents duplicate registrations for the same user, even with concurrent requests.

3 Atomicity Alone Is Not Enough

Transactions are atomic, but without proper locking and constraints, concurrent transactions can still oversubscribe. Rollback on errors to keep data consistent.

Let’s design APIs

© Copyright Intellipaat. All rights reserved.

Functional Requirements

FR-01	Sign in
FR-02	Browse and filter
FR-03	View details
FR-04	Register
FR-05	Confirmation
FR-06	My Registrations
FR-07	Organizer attendee view for owned events

Trace each requirement to a user story, screen, API and acceptance test.

Define the API contract

IntelliPaat

Method	Endpoint	Description
GET	/api/events	Browse events.
GET	/api/events/{id}	Event details.
POST	/api/registrations	Create registration, 201 or 409.
GET	/api/me/registrations	Current user's registrations.
GET	/api/events/{id}/registrations	Attendee list, owning organizer only.



Authentication required for registration *and* private lists.

Concrete Registration Request And Response

IntelliPaat

Registration Request

POST `/api/registrations`

```
{
  "event_id": "550e8400-e29b-41d4-
  a716-446655440000"
}
```

Only **event_id** is sent from the client.
User identity comes from the authenticated context, **not** from a user-supplied ID.



Authenticated User

POST /api/registrations

FastAPI



Create Registration

Use user_id from auth context

Do not accept user_id from request body

201 Created

```
{
  "id": "660e8400-e29b-41d4-
  a716-446655440001",
  "event_id": "550e8400-e29b-41d4-
  a716-446655440000",
  "status": "REGISTERED"
}
```

409 Conflict

```
{
  "detail": "DUPLICATE_REGISTRATION"
}
```

or

```
{
  "detail": "EVENT_FULL"
}
```

401 Unauthorized

```
{
  "detail": "Not authenticated"
}
```

Design An API Contract

IntelliPaat

06:00

Student Registrations API

Define the contract for GET **/api/me/registrations**

Actor	
Auth Required	
Success Response (200) Sample JSON	
Empty State (200) Sample JSON	
Unauthorized (401) Sample JSON	

Organizer Attendees API

Define the contract for organizer to get event attendees

Actor	
Auth Required	
Success Response (200) Sample JSON	
Forbidden (403) When wrong role Sample JSON	
Unauthorized (401) Sample JSON	

Validation Checklist

- [ ] Correct actor and role based access
- [ ] Appropriate authentication requirement
- [ ] Clear success response structure
- [ ] Handles empty state (where applicable)
- [ ] Includes 401 for unauthenticated requests
- [ ] Includes 403 for wrong role (organizer API)
- [ ] Consistent JSON structure and field names
- [ ] Follows REST and schema conventions

Architect-to-backend handoff

IntelliPaat



React + FastAPI + PostgreSQL architecture



UUID-based ERD and constraints



API contracts



Event-row lock protocol for capacity



Authentication and ownership rules



POST /api/registrations • event_id only • 201 success • 409 duplicate or full.



User identity comes from the authenticated session.

Think Like A Backend Engineer

Backend layers and responsibilities

IntelliPaat

1. **Route**



POST /api/registrations

2. **Authentication**



User identity comes from authentication.

3. **Request validation**



JSON body contains event_id only.

4. **Registration service**



Service checks event and business rules.

5. **Database transaction**



Transaction locks event row and protects capacity.

6. **Response**



201 or a specific error.

Explore the Starter FastAPI Repository

IntelliPaat

Project Structure

- campusconnect/
  - app/
    - main.py — Creates FastAPI app and includes routers
    - routes/
      - events.py — Event related endpoints
      - registrations.py — Registration related endpoints
    - schemas/
      - events.py — Pydantic models for request/response validation
      - registrations.py
    - services/
      - events.py — Business logic for events and registrations
      - registrations.py
    - models/
      - event.py — SQLAlchemy database models
      - registration.py
    - db/
      - session.py — Database engine and session
  - tests/ — Test cases for APIs and services
  - migrations/ — Alembic migrations
  - .env.example — Environment variables template
  - README.md — Project overview and setup instructions

Run The Application

```
$ uvicorn app.main:app --reload

INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process
```

Main Application

app/main.py

- Creates the FastAPI application
- Includes events and registrations routers
- Configures middleware and settings

API Routes

routes/events.py

- Endpoints to create, list and manage events

routes/registrations.py

- Endpoints to register for events and view registrations

Database Models

models/event.py

- Event model with id, organizer_id, title, capacity, status

models/registration.py

- Registration model with id, user_id, event_id, created_at

Other Important Files

- **db/session.py** — Database engine and session setup
- **tests/** — Test cases for APIs and services
- **migrations/** — Alembic migrations for database changes
- **.env.example** — Template for environment variables
- **README.md** — Project overview and setup instructions

Configure Database and Migrations

IntelliPaat

1 Use Hosted PostgreSQL in .env

Store your hosted PostgreSQL connection URL in the backend .env file. Keep your credentials private and never share them.

.env (backend)

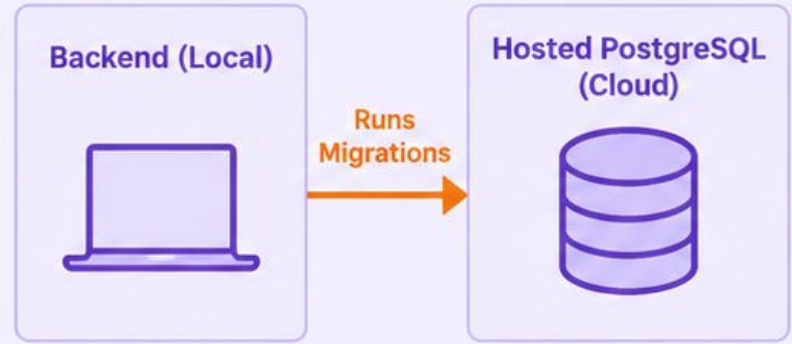
```
DATABASE_URL=postgresql://
campus_user:************@
db-abc123.hosted-postgres.com:5432/
campusconnect
```

2 Run Migrations to Create Tables

Migrations set up the database schema on the hosted PostgreSQL instance.

Terminal (backend)

```
$ alembic upgrade head
INFO  [alembic.runtime.migration]
Running migrations ...
INFO  [alembic.runtime.migration]
Applied 2 revisions.
```



3 Database Schema Created by Migrations

Migrations create the required tables with the following structure.

events

Column	Type	Constraints
id	UUID	Primary Key
organizer_id	UUID	Not Null
title	VARCHAR	Not Null
capacity	INTEGER	Not Null
status	VARCHAR	Not Null

registrations

Column	Type	Constraints
id	UUID	Primary Key
user_id	UUID	Not Null
event_id	UUID	Not Null
created_at	TIMESTAMP	Not Null

4 Seed Initial Events

Seed data populates the database with sample campus events.

Terminal (backend)

```
$ python seed_events.py
Inserted 3 events.
```

Seeded Events

Title	Capacity	Status
AI Build Night	100	open
Robotics Sprint	80	open
Open Source Jam	120	open

DEMO — Inspect the database

IntelliPaat

Query

```
SELECT title, capacity, status
FROM events;
```

Seeded events

title	capacity	status
AI Build Night	50	open
Robotics Sprint	30	open
Open Source Jam	100	open

1. Inspect registrations before a successful API call.
2. Make a successful API call to register for an event.
3. Inspect registrations after the API call. Verify one new row.
4. Repeat the same request and verify no extra row.

Use the provided fixtures and test accounts.

Registration Service Pseudocode



Pseudocode

```
user = get_authenticated_user()
event = db.query(Event).filter_by(id=event_id).with_for_update().first()
if not event or event.status != 'open':
    raise NotFoundError('Event not found or not open')
existing = db.query(Registration).filter_by(user_id=user.id, event_id=event.id).first()
if existing:
    raise ConflictError('DUPLICATE_REGISTRATION')
registered = db.query(func.count(Registration.id)).filter_by(event_id=event.id).scalar()
if registered >= event.capacity:
    raise ConflictError('EVENT_FULL')

try:
    db.add(Registration(user_id=user.id, event_id=event.id))
    db.commit(); return Created(201, 'Registration successful')
except IntegrityError as e:
    db.rollback()
    if is_unique_violation(e, 'registrations_user_id_event_id_key'):
        raise ConflictError('DUPLICATE_REGISTRATION')
    else:
        raise DatabaseError('Could not create registration')
```

1 Get Authenticated User

Retrieve the currently authenticated user from the request context.

2 Lock Event Row

Fetch the event using its ID with row-level lock (FOR UPDATE) to handle concurrent registrations.

3 Reject Missing / Closed Event

If the event does not exist or its status is not 'open', return an error.

4 Check Duplicate Registration

Look for an existing registration for this user and event. If found, return 409 with DUPLICATE_REGISTRATION.

5 Count Seats & Verify Capacity

Count current registrations for the event. If count is greater than or equal to capacity, return 409 with EVENT_FULL.

6 Create Registration (Transaction)

Insert the registration inside a transaction. On IntegrityError, map the unique (user_id, event_id) violation to DUPLICATE_REGISTRATION (409). Roll back and handle other integrity errors separately.
Return 201 on success.

Complete A Backend Guard

IntelliPaat

Hands-On · 10 Minutes

Starting Point: CampusConnect starter service and failing pytest case.

Task: Add duplicate protection OR the full-capacity guard.

Run: pytest tests/test_registrations.py

Deliver: Updated code and test result.

Done When: The targeted test changes from failing to passing.

```
1  def register(...):
       # TODO: complete the selected guard
```

```
$ pytest tests/test_registrations.py
F.........
```



```
$ pytest tests/test_registrations.py
..........
```

Production-Safe Guard Pattern

IntelliPaat

1



Database:
UNIQUE(user_id, event_id)

2



Transaction: Lock event row, then count registrations.

3



4



Full Event:
Roll back and return EVENT_FULL (409).

Duplicate Constraint:
Roll back and return DUPLICATE_REGISTRATION (409).

Other Errors:
Roll back and handle separately.



Frontend:
Display the server's result.

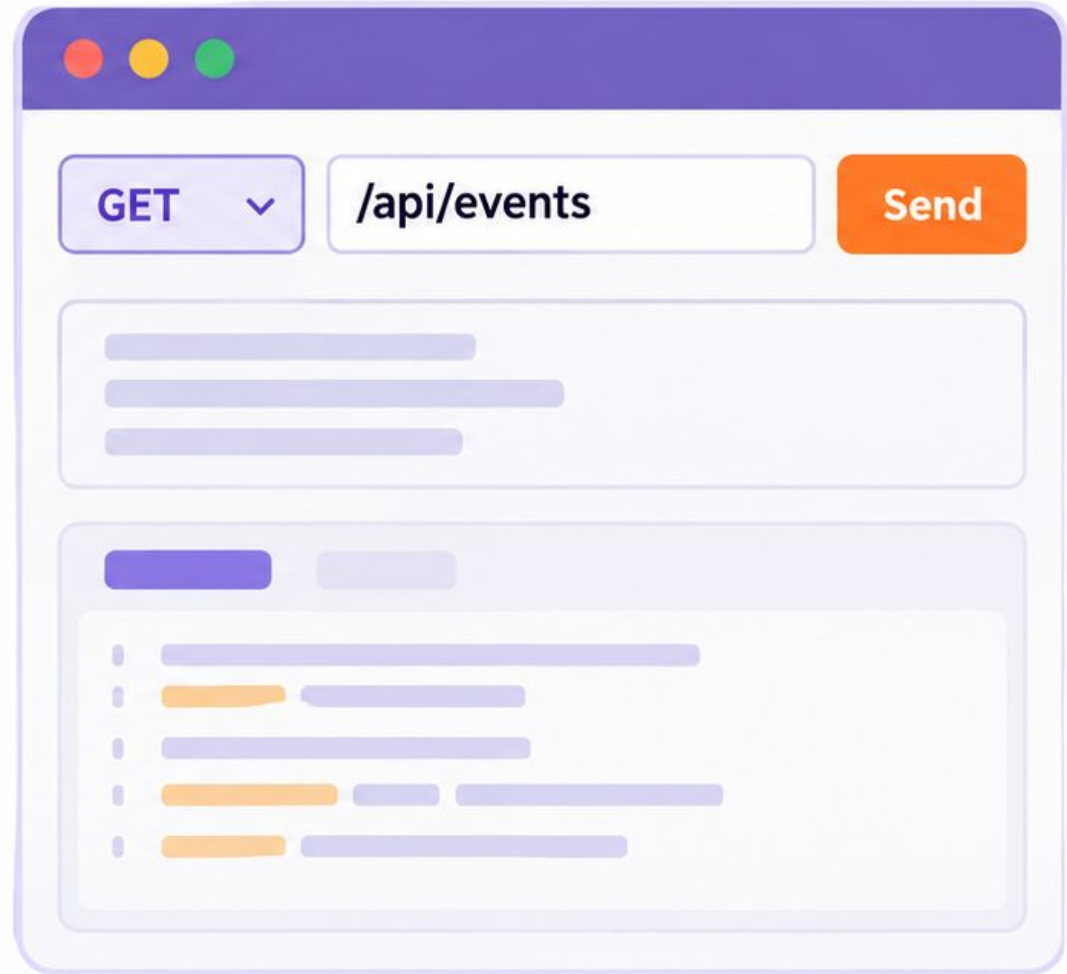
Test API Behaviors In Swagger

IntelliPaat

Example API Tests

GET /api/events	→ 200
GET /api/events/{id}	→ 200 or 404
POST /api/registrations	→ 201
Repeat Registration	→ 409
Full Event	→ 409
Missing Authentication	→ 401
Unauthorized Attendee Access	→ 403

Compare response body and stored rows.



API Test Quest

IntelliPaat

Hands-On · 7 Minutes

Use Your Assigned Test Account

1. List events and open one event. ☐
2. Register and capture the 201 response. ☐
3. Verify the stored registration. ☐
4. Repeat and capture the 409 response. ☐

Bonus: Test organizer attendee access. ☐

Deliver: Response evidence and row check. ☐



Reliable Shared Backend

IntelliPaat

Source Repository

Clone to explore the implementation.

Hosted API

Connect the frontend to the instructor-provided HTTPS API URL.



Handoff Package

OpenAPI docs · Test accounts · Fixtures · Error codes



Database credentials stay on the backend.

Think Like A UI/UX Designer

User Friction Guides Design

IntelliPaat

Scattered Links	→	One Event Catalog
Unclear Eligibility	→	Visible Rules And Seat Status
Repeated Forms	→	Ask Only For Necessary Data
Uncertain Outcome	→	Confirmation And My Registrations
Organizer Confusion	→	Attendee View For Owned Events

Information Architecture And Screen Map



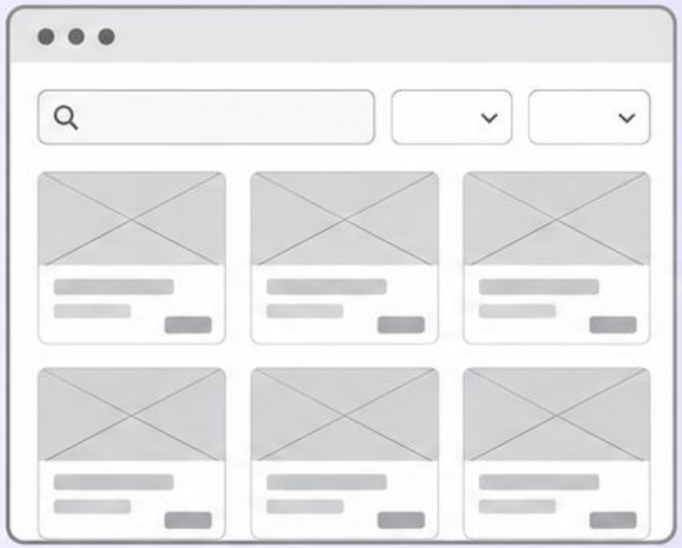






Low-Fidelity Wireframes Come First

IntelliPaat



Explore Events

Search · Filters · Event Cards



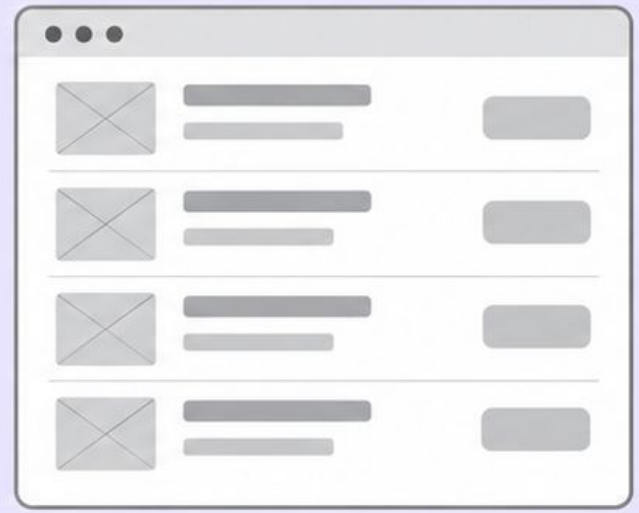
Event Details

Facts · Eligibility · Seats · Register



Confirmation

Outcome · Reference · Next Step



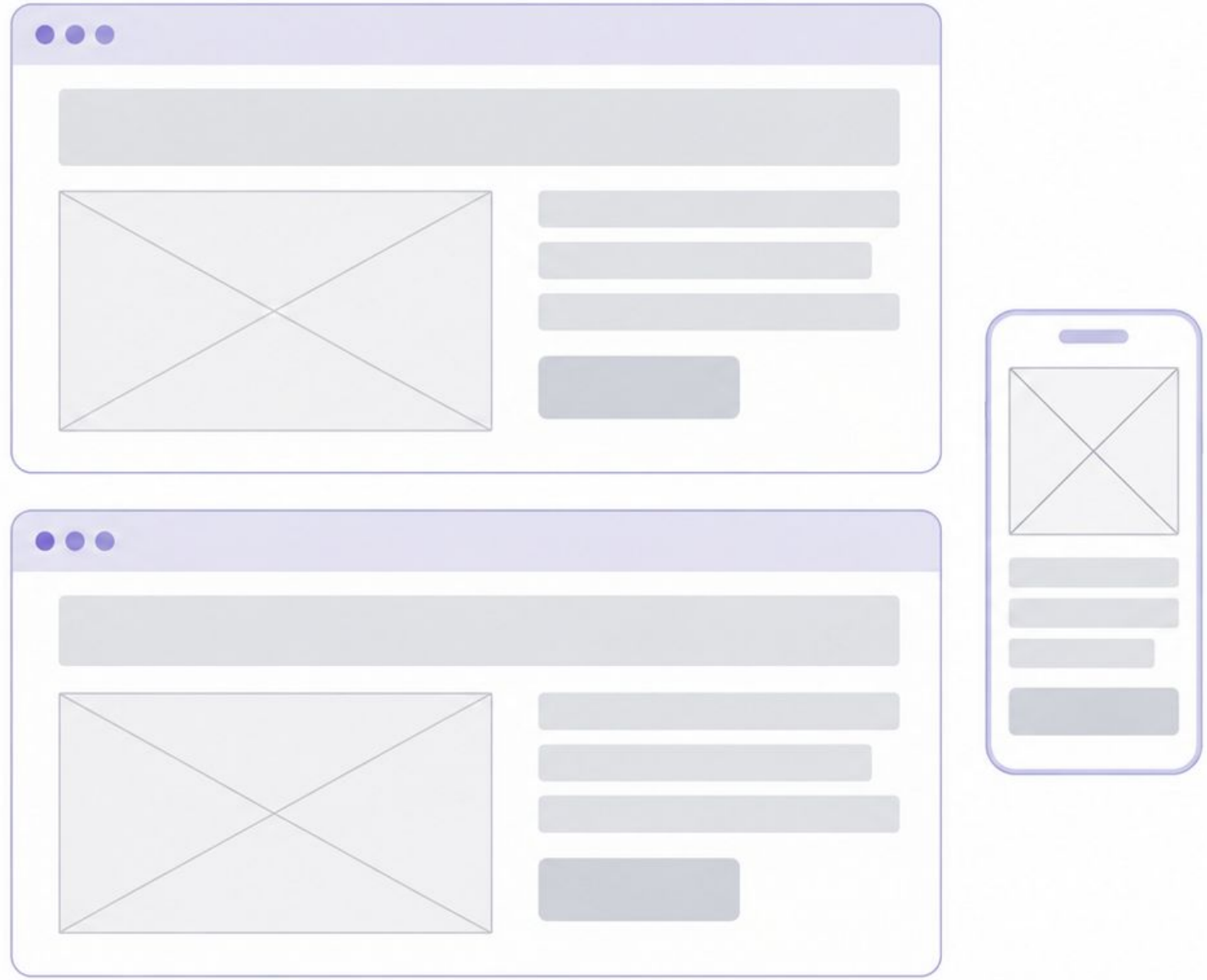
My Registrations

Event List · Registration Status

Solve The Friction With Wireframes

IntelliPaat

- Hands-On · 10 Minutes
- Design Explore Events And Event Details
- Mark: Primary action, eligibility, seat status and mobile navigation.
- Annotate three decisions that reduce user friction.
- Deliver: Two wireframes with annotations.
- Check: Can a partner find the next action in five seconds?



Why the wireframe works

IntelliPaat

1. Clear first action.



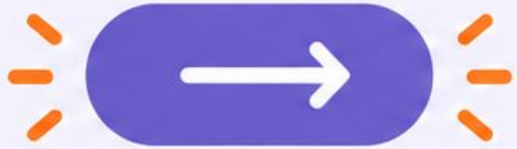
2. Filters near event results.



3. Event facts above the fold.



4. One strong Register CTA.



5. Clear signed-out, closed and full states.



6. No unnecessary registration fields.



CampusConnect visual direction

Credible • Energetic • Accessible • Contemporary

Navy	Violet	Background	Slate	Coral
#111827	#6D5EF8	#F8FAFC	#475569	#F97360

CampusConnect

Find your next campus experience.

Use coral sparingly.

Component board

IntelliPaat

Button

Default	Button
Focus	Button
Loading	Button
Disabled	Button
Error	Button
Success	Button

Input

Default	Enter text
Focus	Enter text
Loading	Enter text
Disabled	Enter text
Error	Enter text
Success	Enter text

Card



Card title

Short description goes here.

Button Button

Chip

Default	Chip	Loading	Chip	Error	Chip
Focus	Chip	Disabled	Chip	Success	Chip

Status

Default	Neutral	Disabled	Disabled
Focus	In Progress	Error	Error
	Loading	Success	Success

Design tokens

Radius

Shadow

Spacing (8px rhythm)



Your design tokens

1 Choose your colors

Main | Secondary | Background | Text | Error

2 Define your type scale

H1

H2

H3

Body

Small

3 Set spacing and radius

Spacing (xs)

Spacing (sm)

Spacing (md)

Spacing (lg)

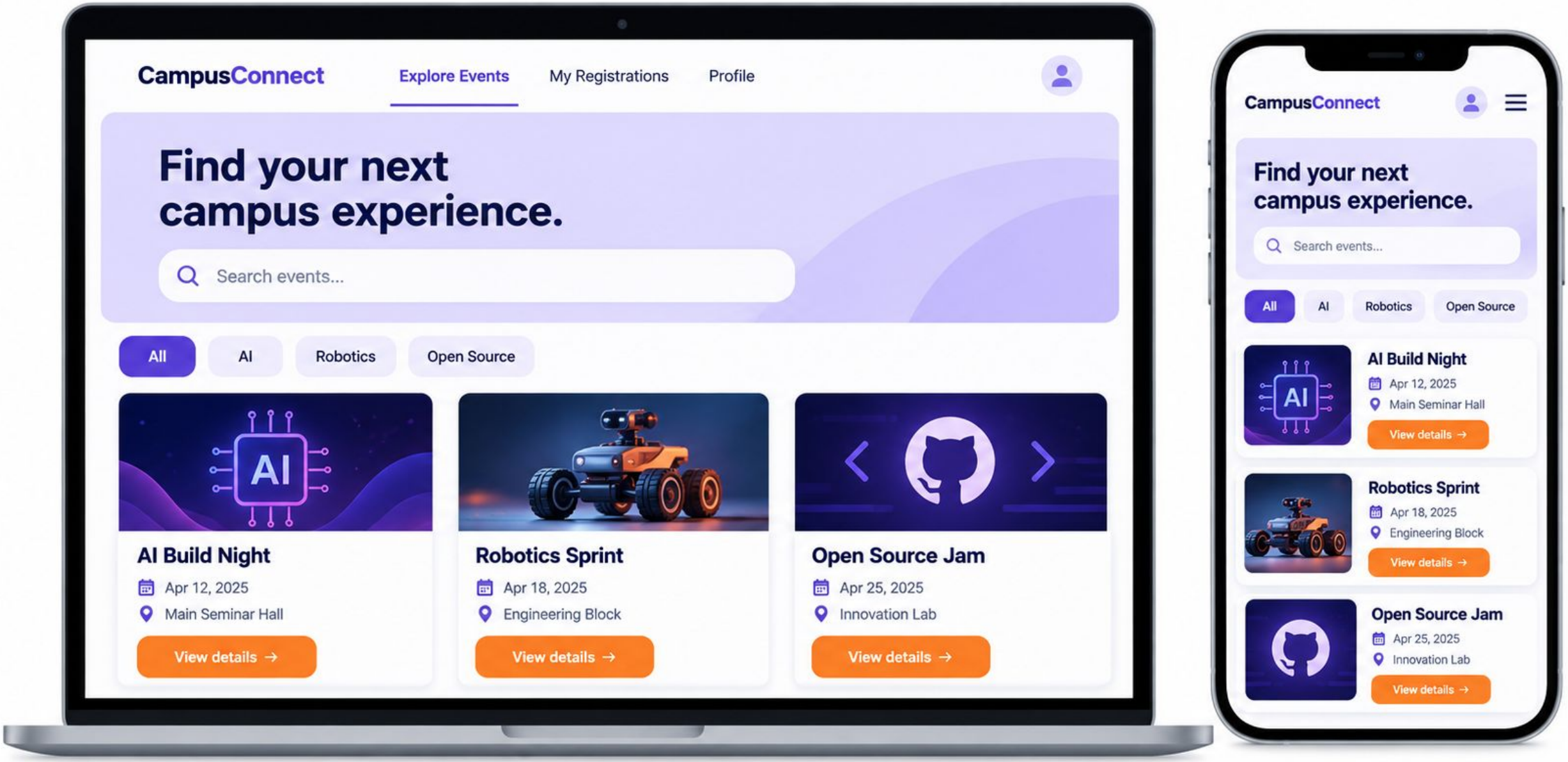
Radius

4 Design 3 components

5 Explain your choices

Explore Events

IntelliPaat



Event Details

CampusConnect Home Explore Events My Registrations Search events...

Explore Events / AI Build Night



AI Build Night

12 November | 5:00 PM | Main Campus | Open to students | 8 seats left

A hands-on evening to build, learn and collaborate on real-world AI projects with fellow students. Bring your ideas and get guidance from peers and mentors.

Register Now

CampusConnect Home Explore Events My Registrations Search events...

Explore Events / AI Build Night



AI Build Night

12 November | 5:00 PM | Main Campus | Open to students | 8 seats left

A hands-on evening to build, learn and collaborate on real-world AI projects with fellow students. Bring your ideas and get guidance from peers and mentors.

Sign in to register

CampusConnect Home Explore Events My Registrations Search events...

Explore Events / AI Build Night



AI Build Night

12 November | 5:00 PM | Main Campus | Open to students | Event Full

A hands-on evening to build, learn and collaborate on real-world AI projects with fellow students. Bring your ideas and get guidance from peers and mentors.

Browse other events

Confirmation and history

IntelliPaat

CampusConnect

You're registered!

AI Build Night

Status: Confirmed

Reference: REG-EXAMPLE

View My Registrations

CampusConnect

My Registrations

AI Build Night Confirmed

EVENT_FULL

This event is full.

Browse other events

MISSION 04C — Professional screen challenge

IntelliPaat

8 minutes.

Turn one wireframe into high-fidelity Event Details.

Show three states:
Default; Full; Already registered.

Reuse your design tokens and components.

Deliver: one screen with state variants.



Default

Full

Already registered

IntelliPaat

Design-to-code handoff



Sitemap



Annotated wireframes



Design tokens



Reusable components



Responsive breakpoints



Microcopy and API-driven states

Defend one UX decision using a user friction.



THINK LIKE A FRONTEND ENGINEER

Break screens into React components

- **AppShell**
  - Navbar
- **EventListPage**
  - SearchBar
  - FilterChips
  - EventCard
- **EventDetailsPage**
  - RegisterButton
- **MyRegistrationsPage**
  - StatusBadge
  - Alert

AppShell → Navbar

IntelliPaat Events My Registrations Profile

EventListPage → SearchBar, FilterChips, EventCard

Search events...

All | Upcoming | Online | In-Person



React Development Workshop

Apr 20, 2025

Online

EventDetailsPage → RegisterButton



React Development Workshop

Apr 20, 2025 Online

Register

MyRegistrationsPage → StatusBadge, Alert

React Development Workshop

Apr 20, 2025

Registered

You are successfully registered for this event.

From static cards to live events

IntelliPaat

React page



GET /api/events

GET /api/events

JSON

```
[
  {
    "id": "event-uuid",
    "title": "AI Build Night"
  }
]
```

(Illustrative response)

EventCard list

AI Build Night

AI Build Night

Loading



Results

AI Build Night

AI Community Meetup

Design with AI

No results

No events found

Error

Failed to load events

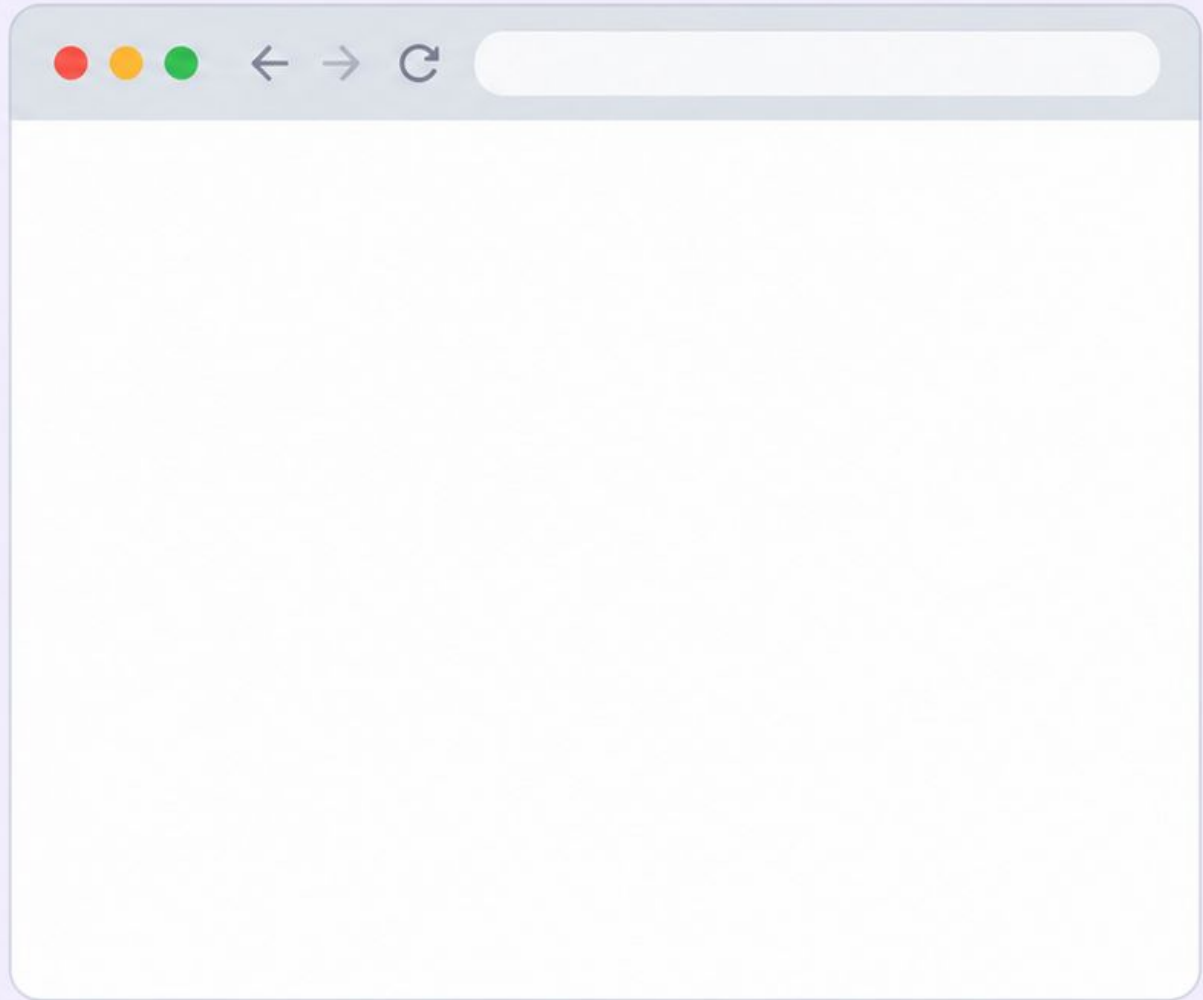
Try again

Avoid showing stale results after filter changes.

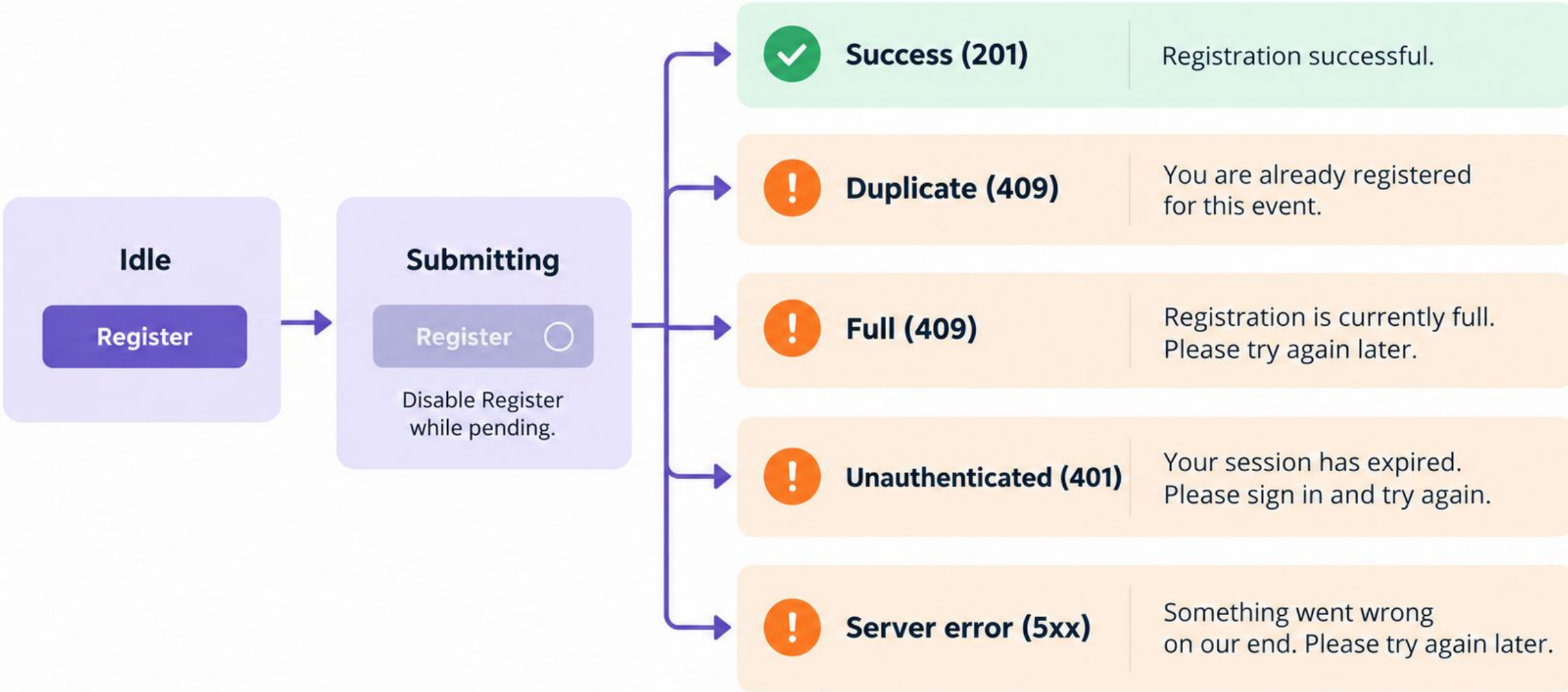
MISSION 05A — Build the live event listing

1. **12 minutes**
   Open frontend starter.
2. Set the provided public API base URL.
3. Fetch GET /api/events.
4. Render title, date and category from the API.
5. Handle loading, empty and error states.
6. **Done when:** seeded events appear in your UI.
7. Never place database credentials in frontend code.

Browser checklist



Register a state machine



MISSION 05B — Connect Register Now

IntelliPaat

1 12 minutes

Event Details



Register Now

2 Call POST /api/registrations from Event Details

POST /api/registrations

```
{
  "event_id": "event-uuid"
}
```

Send session authentication.

3 Show progress, success and conflict errors

Registering...

Registration successful!

You are already registered for this event.

4 Refresh My Registrations

My Registrations



Registered

5 Done when:

- One click creates one registration and the result appears in history.
- Request body has no user_id.

Event Registration Flow



8 minutes

Swap QA partners

Scenario	Pass	Fail	Evidence
Happy path			
Duplicate registration			
Full event			
Signed out			
Mobile viewport			
Keyboard navigation			



Collect a screenshot or API response.



Match results to acceptance criteria.

One feature across five roles

CampusConnect event registration — FR-04

IntelliPaat



Product Manager

User story and acceptance criteria.



Architect

API contract and data model.



Backend Engineer

Transaction and capacity guard.



UI/UX Designer

Clear action and states.



Frontend Engineer

Working React interaction.



Verify the journey with an end-to-end test.

Day 1 evidence checklist

IntelliPaat

1.  BRD and complete MVP FRD ☐

2.  Architecture, ERD and API contracts ☐

3.  Tested API evidence ☐

4.  Wireframes, tokens and high-fidelity screen ☐

5.  Frontend link or local demo ☐

6.  Test results and one design trade-off ☐

Day 2: Source code to hosted product



- Commit, branch and pull request
- Deploy frontend to Vercel or another host
- Reuse or deploy backend
- Configure environment and CORS
- Validate public HTTPS URL

GitHub stores source code.

Hosting runs the product.